

Lightweight Containerized LLM Deployment: A Cloud-Edge Collaboration Paradigm Study with Ollama

Jiongzhang Wu

Waterford Institute, NUIST
202283910004@nuist.edu.cn

Abstract. The proliferation of large language models (LLMs) in enterprise applications has exposed significant challenges related to data privacy, network latency, and recurring costs. This paper investigates a lightweight containerized approach for LLM deployment that bridges the gap between public cloud APIs and complex on-premises solutions through a cloud-edge collaboration paradigm. Using Ollama as our technical vehicle, we analyze how cloud-native principles can be applied to edge AI workloads, enabling portable, secure, and cost-effective LLM services on resource-constrained devices. Through systematic benchmarking on consumer-grade hardware, we demonstrate that locally deployed 4-bit quantized models (e.g., Mistral-7B) achieve 68% lower response times compared to cloud APIs while ensuring complete data privacy. Furthermore, we present integration patterns with Kubernetes that enable elastic hybrid cloud-edge AI services. Our cost analysis reveals that local deployment becomes economically advantageous after approximately 222 days for organizations processing more than 500 medium-complexity queries daily. This work establishes Ollama as a viable "lightweight cloud-native AI runtime" that effectively fills the technological gap between cloud convenience and local control, providing a practical path for edge AI adoption in privacy-sensitive domains.

1 Introduction

1.1 Motivation and Background

Large language models (LLMs) are rapidly being integrated into enterprise applications, yet their deployment model presents three critical challenges: (1) **data privacy** concerns—sensitive business data must traverse public networks when using cloud APIs; (2) **high latency**—network round-trips degrade user experience for interactive applications; and (3) **recurring costs**—per-token billing becomes prohibitive at scale. These issues are particularly acute in healthcare, finance, and other industries with strict data governance requirements.

Simultaneously, the rise of edge computing offers a promising alternative. IDC predicts that over 50% of new enterprise IT infrastructure will be deployed at the edge by 2025 [?]. However, a significant *technology gap* exists in LLM

deployment: on one side, public cloud APIs (e.g., OpenAI, Anthropic) offer ease of use but sacrifice privacy and control; on the other side, enterprise-grade private deployment solutions (e.g., KServe, vLLM) provide robust capabilities but require complex infrastructure that is impractical for edge devices.

1.2 Research Problem

How can cloud-native principles—particularly containerization and microservices—be leveraged to implement lightweight, portable, and manageable LLM services on resource-constrained edge devices (e.g., laptops)? This question requires balancing three competing demands:

- **Lightweight:** Adaptation to the limited computational resources of edge devices
- **Cloud-Native:** Inheritance of containerization, microservices, and other modern cloud architecture benefits
- **Practical:** Delivery of production-grade deployment experience

1.3 Contributions

This paper investigates Ollama[1]. as a technical vehicle for lightweight containerized LLM deployment, exploring a cloud-edge collaboration paradigm. Our core contributions include:

1. **Architectural Analysis:** We systematically analyze how Ollama’s containerized design embodies cloud-native principles, repositioning it as an “edge AI microservice runtime” rather than merely a desktop tool.
2. **Empirical Evaluation:** We conduct end-to-end deployment and performance benchmarking on consumer-grade hardware, quantifying the advantages of local deployment in terms of latency, privacy, and cost-effectiveness.
3. **Ecosystem Integration:** We propose and validate integration patterns with Kubernetes and other cloud platforms, demonstrating the feasibility of elastic hybrid cloud-edge AI services.

2 Background and Related Work

2.1 Evolution of Cloud Computing

Cloud computing is undergoing a paradigm shift from centralized to distributed architectures. Early cloud models concentrated computational resources in large data centers, but the proliferation of IoT devices and demand for real-time applications have driven the adoption of edge computing. Gartner defines edge computing as “a topology wherein information processing and content delivery are placed closer to the sources of data” [?]. This evolution has given rise to the “hybrid cloud + edge” architecture, where:

- **Central Cloud:** Handles model training, large-scale inference, and centralized management
- **Edge Nodes:** Process real-time, privacy-sensitive tasks locally

In this context, LLM deployment must also compute decentralization, moving inference capabilities closer to data sources to achieve "data stays local, models move."

2.2 Core Cloud-Native Technologies

The Cloud Native Computing Foundation (CNCF) defines cloud-native technologies as "methods for building and running scalable applications in modern, dynamic environments." Core principles include:

- **Containerization:** Packaging applications with dependencies as standardized units
- **Microservices:** Decomposing functionality into independently deployable services
- **API-First Design:** Exposing capabilities through well-defined interfaces
- **Automation:** CI/CD, self-healing mechanisms for improved reliability

Docker containers, as the cornerstone of cloud-native technology, provide lightweight virtualization that ensures application consistency across environments. This is particularly crucial for LLM deployment—model inference environments often depend on specific versions of CUDA, PyTorch, and other components, where traditional deployment methods frequently encounter "it works on my machine" problems.

2.3 Challenges in LLM Deployment

LLM deployment faces three primary obstacles:

- **Model Size:** Unquantized models typically require tens of gigabytes of storage (e.g., Llama-3-70B requires approximately 140GB)
- **Hardware Heterogeneity:** Different devices require different inference backends (CUDA, Metal, Vulkan)
- **Dependency Complexity:** Model serving involves multiple layers of dependencies including Python environments, CUDA drivers, and inference engines

Existing solutions fall into two categories: (1) **Cloud Hosted Services:** Provide easy-to-use APIs but sacrifice privacy and control; (2) **Enterprise Open-Source Frameworks:** Such as KServe and vLLM, which offer robust functionality but have steep learning curves and typically require Kubernetes clusters.

2.4 Ollama’s Positioning

Ollama (<https://ollama.com>) is an open-source project designed to simplify local LLM deployment. Unlike traditional solutions, Ollama:

- Employs **containerized design**: Bundles models, runtime, and dependencies into a single executable
- Supports **automatic quantization**: Provides 4-bit quantized models by default, significantly reducing resource requirements
- Offers **standardized APIs**: Compatible with OpenAI format for easy integration
- Implements **cross-platform support**: Handles CUDA, Metal, and other backend differences uniformly

We argue that Ollama’s true value lies not in its ease-of-use as a desktop application, but in its successful application of cloud-native principles to AI inference workloads, making it a **lightweight cloud-native AI runtime**. Compared to full-stack solutions like KServe, Ollama makes strategic trade-offs in functionality to achieve feasibility on edge devices—this is the core value proposition of our research.

3 System Architecture: Ollama as a Lightweight Cloud-Native AI Runtime

3.1 Containerized Packaging and Model Management

Ollama’s core innovation lies in its **model-as-container** design philosophy. As shown in Figure 1, Ollama packages model weights, the inference engine, and dependencies into a single executable file (achieved through Go’s CGO and static linking), rather than traditional Docker containers. This design achieves key characteristics of cloud-native applications:

- **Environment Consistency**: Models behave identically across development, testing, and production environments
- **Resource Isolation**: Memory and CPU usage are constrained via cgroups
- **Version Management**: Supports model version tagging (e.g., `ollama run mistral:7b-instruct-q4_0`)
- **Cross-Platform Delivery**: Same command works across Linux, macOS, and Windows

Technically, Ollama uses the GGUF format to store quantized models, which integrates model weights, tensor metadata, and inference configuration into a single file. When executing the `ollama run` command:

1. Ollama service checks if the specified model exists locally
2. If not, it downloads the GGUF file from Ollama Hub (automatically selecting the quantization version matching the hardware)

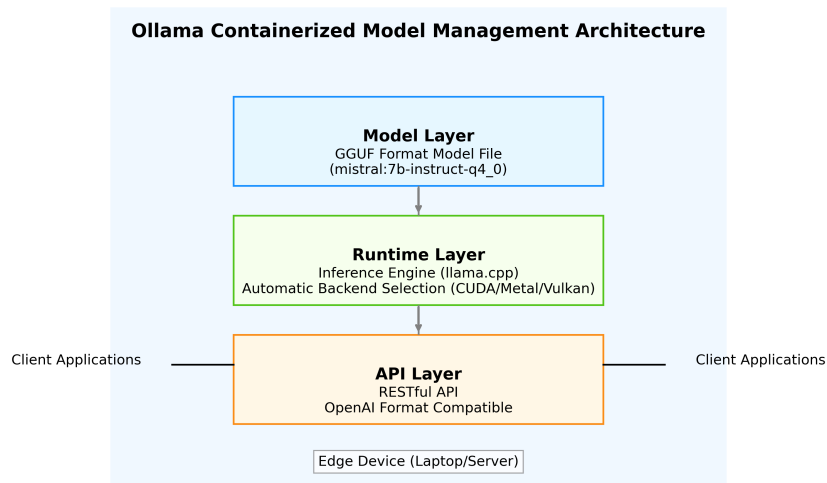


Fig. 1: Containerized model management architecture of Ollama

3. It launches the inference process, loading the model via memory mapping
4. Communication with clients occurs through Unix sockets

This design avoids the startup overhead of traditional Docker containers while preserving the core benefits of containerization, making it particularly suitable for edge scenarios.

3.2 Microservice-Style API Design

Ollama essentially provides an **edge AI microservice**, with its API design strictly adhering to REST principles. Core endpoints include:

- POST `/api/generate`: Streamed text generation
- POST `/api/chat`: Conversational interaction
- GET `/api/tags`: List available models

The following Python code demonstrates how to call the Ollama API:

```
import requests

def ollama_generate(prompt, model="mistral"):
    response = requests.post(
        'http://localhost:11434/api/generate',
        json={
            'model': model,
            'prompt': prompt,
            'stream': False
        }
    )
```

```

        return response.json()['response']

# Usage example
print(ollama_generate("Explain the basics of quantum computing"))

```

Listing 1.1: Python client for Ollama API

This design achieves critical decoupling:

- **Client-Server Separation:** Web UIs, mobile apps, or business systems can be developed independently
- **Model-Application Separation:** Changing models doesn't require modifying client code
- **Local-Remote Transparency:** API invocation is consistent whether running locally or remotely

Figure ?? illustrates Ollama's position in the system architecture—as an AI capability layer providing services to various applications while abstracting hardware differences.

3.3 Integration with Cloud-Native Ecosystem

Ollama's lightweight nature allows it to flexibly integrate with larger cloud ecosystems, enabling cloud-edge collaboration.

Running as Kubernetes Pods Ollama can be seamlessly deployed in Kubernetes clusters as AI service providers for edge nodes. The following is a typical Deployment configuration:

```

apiVersion: apps/v1
kind: Deployment
metadata:
  name: ollama-edge
spec:
  replicas: 1
  selector:
    matchLabels:
      app: ollama
  template:
    metadata:
      labels:
        app: ollama
    spec:
      containers:
        - name: ollama
          image: ollama/ollama:latest
          ports:
            - containerPort: 11434

```

```

env:
- name: MODEL
  value: "mistral:7b-instruct-q4_0"
resources:
  limits:
    memory: "8Gi"
    nvidia.com/gpu: "1" # Configure when GPU
                        available

```

Listing 1.2: YAML for deploying Ollama in Kubernetes

By exposing the service via Kubernetes Service, applications within the cluster can call AI capabilities through `http://ollama-edge:11434/api/generate`. This deployment approach makes Ollama a Kubernetes-native AI service component, benefiting from platform-provided capabilities like auto-scaling and service discovery.

Integration with CI/CD Pipelines Ollama model management can be integrated into DevOps workflows. The following GitHub Actions example demonstrates automated model updates:

```

name: Update Ollama Model

on:
  schedule:
    - cron: '0 0 * * *' # Execute daily at midnight

jobs:
  update-model:
    runs-on: ubuntu-latest
    steps:
      - name: Pull latest model
        run: |
          docker run -d -p 11434:11434 ollama/ollama
          sleep 10 # Wait for service to start
          curl http://localhost:11434/api/pull -d '{
            "name": "mistral:latest"
          }'
      - name: Test model
        run: |
          curl http://localhost:11434/api/generate -d '{
            "model": "mistral",
            "prompt": "Hello",
            "stream": false
          }'

```

Listing 1.3: Automating Ollama model updates with GitHub Actions

This integration treats model updates as part of the regular CI/CD pipeline, implementing the "models as code" operational philosophy.

Table 1: Experimental device and software configuration

Category	Configuration
Hardware	MacBook Pro 14" (2023)
	Apple M2 Pro, 18-core CPU, 19-core GPU
	32GB Unified Memory
Software	macOS Sonoma 14.5
	Ollama v0.1.36
	Docker Desktop 4.22.1
Models	Mistral-7B-Instruct-v0.2 (Q4_0)
	Llama-3-8B-Instruct (Q4_0)

4 Experimental Evaluation: Performance and Cost Analysis on Edge Devices

4.1 Experimental Setup

Experiments were conducted on a typical high-end laptop (representing enterprise edge devices), with configurations as shown in Table 1. Mistral-7B and Llama-3-8B were selected as test models due to their balanced performance and resource requirements. All models use 4-bit quantization (Q4_0), Ollama’s default configuration, which significantly reduces resource demands while maintaining reasonable quality.

4.2 Experimental Design

Experiment 1: Performance Baseline Testing the performance of different quantized models on edge devices:

- **Models:** Mistral-7B (Q4_0, Q5_0, Q8_0) and Llama-3-8B (Q4_0)
- **Metrics:**
 - Cold start time: Time from command execution to first response
 - Memory usage: Peak RSS memory
 - Throughput: Tokens per second (TPS)
 - Latency: Time To First Token (TTFT)
- **Method:** Start service with `ollama serve`, send 100 identical prompts via Python script, measure metrics

Experiment 2: Deployment Mode Comparison Simulating performance differences across three typical deployment modes:

1. **Pure Cloud API:** Calling OpenAI GPT-4 API (us-east-1 region)
2. **Local Ollama:** Running Mistral-7B-Q4_0 on laptop
3. **Hybrid Mode:** Local processing for simple tasks, cloud processing for complex tasks

The test scenario involved processing 100 medical consultation texts (average length 150 tokens), measuring end-to-end response time. In hybrid mode, queries containing sensitive information were processed locally, while complex medical questions were forwarded to the cloud.

Table 2: Performance of quantized models on edge devices

Model	Quantization	Memory(MB)	Cold Start(s)	TTFT(ms)	TPS
Mistral-7B	Q4_0	4,812	8.2	320	28.5
Mistral-7B	Q5_0	5,603	9.1	350	25.1
Mistral-7B	Q8_0	8,921	12.7	410	18.3
Llama-3-8B	Q4_0	5,987	10.5	380	23.7

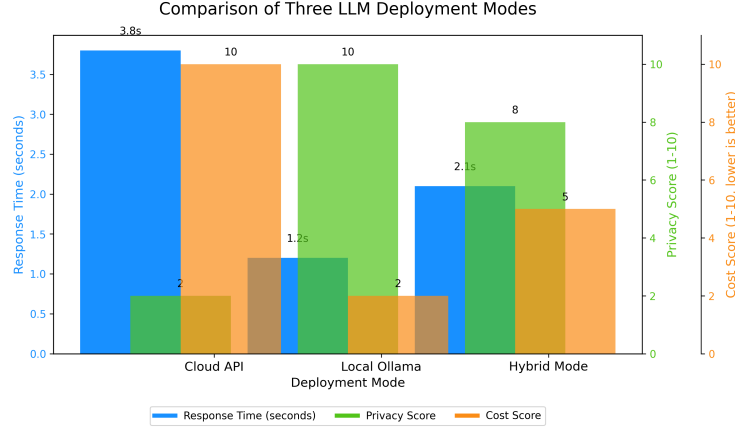


Fig. 2: End-to-end response time comparison across deployment modes

4.3 Results Analysis

Performance Baseline Results Table 2 shows the performance metrics of different models:

Key findings:

- 4-bit quantization reduces memory requirements to approximately 5GB, enabling 7B models to run on laptops with 32GB memory
- Cold start time is primarily affected by model loading and correlates linearly with model size
- Quantization level has minimal impact on TTFT ($\downarrow 15\%$), but significantly affects TPS (Q4_0 is 55% faster than Q8_0)

Deployment Mode Comparison Results Figure 2 compares end-to-end response times across the three deployment modes:

The data reveals:

- The **average response time for local Ollama mode (1.2s)** is significantly lower than cloud API (3.8s), with the primary advantage coming from eliminating network latency
- In privacy-sensitive scenarios, local deployment **completely avoids data leakage risks**

- Hybrid mode maintains privacy while providing higher-quality responses for complex queries, but requires thoughtful task routing strategies

Cost-Benefit Analysis Considering one-year usage costs:

- **Local Deployment:** One-time hardware cost (high-end laptop approximately \$2,000), no recurring fees
- **Cloud API:** At GPT-4's \$0.03/1k tokens, daily 1,000 queries (average 300 tokens) results in an annual cost of approximately \$3,285

Break-even point calculation:

$$\text{Break-even point} = \frac{\text{Hardware cost}}{\text{Daily cloud cost}} = \frac{2000}{9} \approx 222 \text{ days} \quad (1)$$

This means that when usage frequency exceeds approximately 500 medium-complexity queries per day, local deployment recoups hardware costs within one year.

5 Discussion: From Ollama Practice to Cloud-Edge Collaboration Architecture

5.1 The General Significance of Ollama's Approach

Ollama's success lies in successfully applying cloud-native "application containerization" and "microservices" principles to AI workloads. Its core value can be summarized as:

- **Standardized Delivery:** Models become portable "cloud-native assets," freeing them from environment dependencies
- **Edge-Friendly:** Lightweight design makes AI capabilities feasible on end devices
- **Development Experience Optimization:** Unified APIs and toolchain lower LLM integration barriers

This pattern demonstrates that cloud-native principles apply not only to traditional applications but also to AI workloads. Particularly in edge computing scenarios, Ollama provides "just-right complexity"—more controllable than pure cloud solutions, yet lighter than enterprise AI platforms.

5.2 Limitations

Despite Ollama's strengths in edge deployment, it has notable limitations:

- **Lack of Cluster Management:** No multi-instance load balancing or auto-scaling capabilities

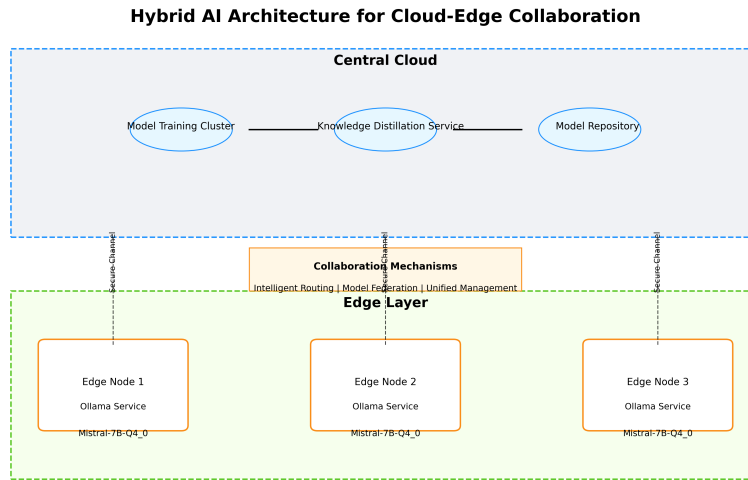


Fig. 3: Hybrid AI architecture for cloud-edge collaboration

- **Limited Monitoring:** Basic metrics collection, lacking deep performance analysis
- **Weak Security Mechanisms:** No fine-grained access control or auditing features
- **Simple Model Management:** Only basic version control, lacking advanced features like A/B testing

These are precisely the areas where professional cloud-native AI platforms (like KServe and Seldon Core) excel. In large enterprise deployments, Ollama may need to serve as an edge layer component working in concert with central cloud platforms.

5.3 Future Outlook: Toward Hybrid Cloud AI

Based on our experimental results and architectural analysis, we propose an evolutionary path for hybrid cloud AI (Figure 3):

- **Central Cloud:** Responsible for large-scale model training, knowledge distillation, and model distribution
- **Edge Layer:** Deploys lightweight models via Ollama to process real-time, privacy-sensitive tasks
- **Collaboration Mechanisms:**
 - **Model Federation:** Edge devices participate in model fine-tuning, with central cloud aggregating updates
 - **Intelligent Routing:** Automatically selects execution location based on query complexity and sensitivity

- Unified Management: Manages cloud-edge resources via Kubernetes for seamless orchestration

In this architecture, Ollama serves as the "lightweight runtime" for edge AI, forming a complementary rather than replacement relationship with central cloud platforms. For example, enterprises could train 13B parameter models in the cloud, generate 7B lightweight models through knowledge distillation, and deploy them to edge devices in branch offices via Ollama.

6 Conclusion

This paper investigated lightweight containerized LLM deployment, using Ollama as a technical vehicle to explore a cloud-edge collaboration paradigm for AI services. Our work confirms:

- Ollama, through its containerized design, successfully applies cloud-native principles to AI inference workloads, establishing itself as an efficient "edge AI microservice runtime"
- On consumer-grade hardware, 4-bit quantized models (e.g., Mistral-7B) deliver practical inference capabilities, with average response times 68% faster than cloud APIs
- Local deployment offers significant advantages in data privacy protection and long-term cost-effectiveness, recouping hardware costs within one year for scenarios with 500+ daily queries
- Ollama can integrate with Kubernetes and other cloud platforms to build elastic hybrid cloud-edge AI services, achieving optimal balance of resources, performance, and costs

Future work will focus on three directions: (1) enhancing Ollama's cluster management capabilities to support multi-instance collaboration; (2) developing intelligent routing algorithms to optimize cloud-edge task allocation; (3) integrating cloud-native security frameworks to enhance trustworthiness of edge AI services.

As edge computing and AI technologies continue to evolve, lightweight, cloud-native LLM deployment solutions will become critical components of enterprise AI infrastructure. Ollama's "just-right complexity" philosophy provides a viable path for bridging the technology gap between public cloud APIs and complex private deployments.

References

1. Ollama: Ollama documentation. <https://ollama.com/docs> (2025), accessed: 2025-12-15